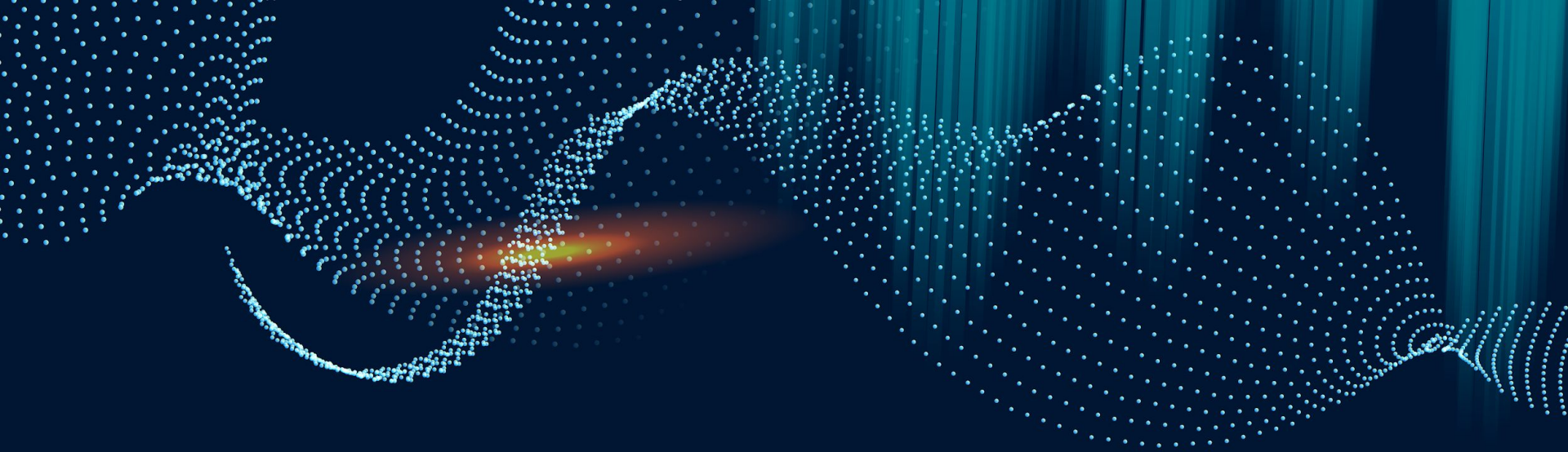
UP
Universidad
de Palermo

INGENIERIA REVERSA

Clase 7

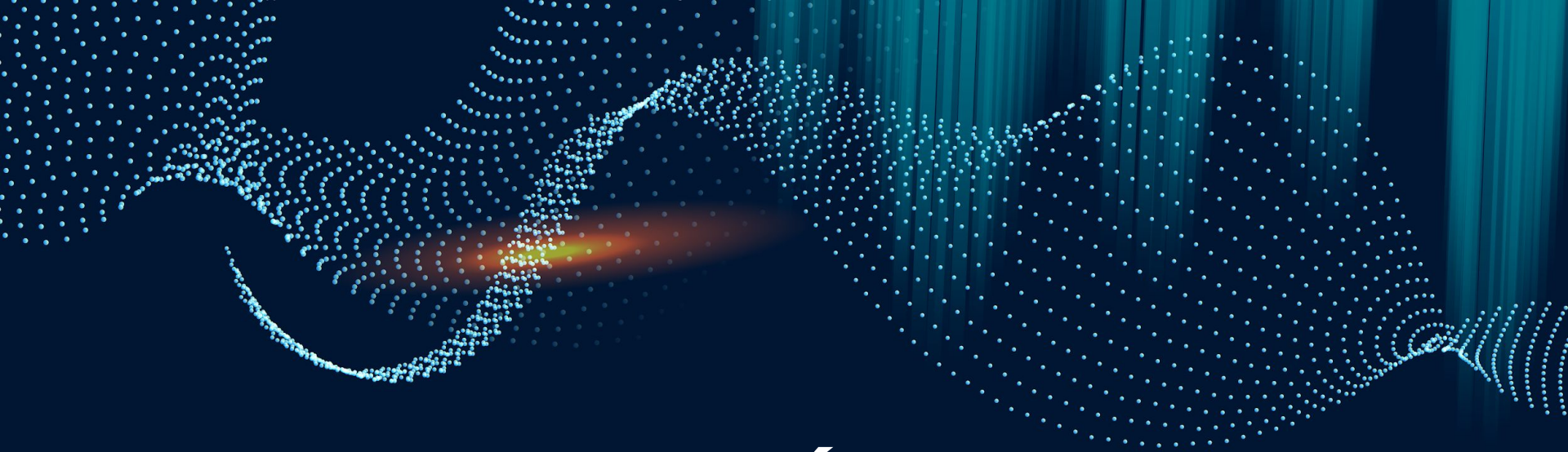
Universidad de Palermo



00 TEMAS

Lo que vamos a ver en la clase

1. Estático Avanzado
2. Descompilar y Desensamblar
3. Assembly
4. Estructura de PE
5. Herramientas

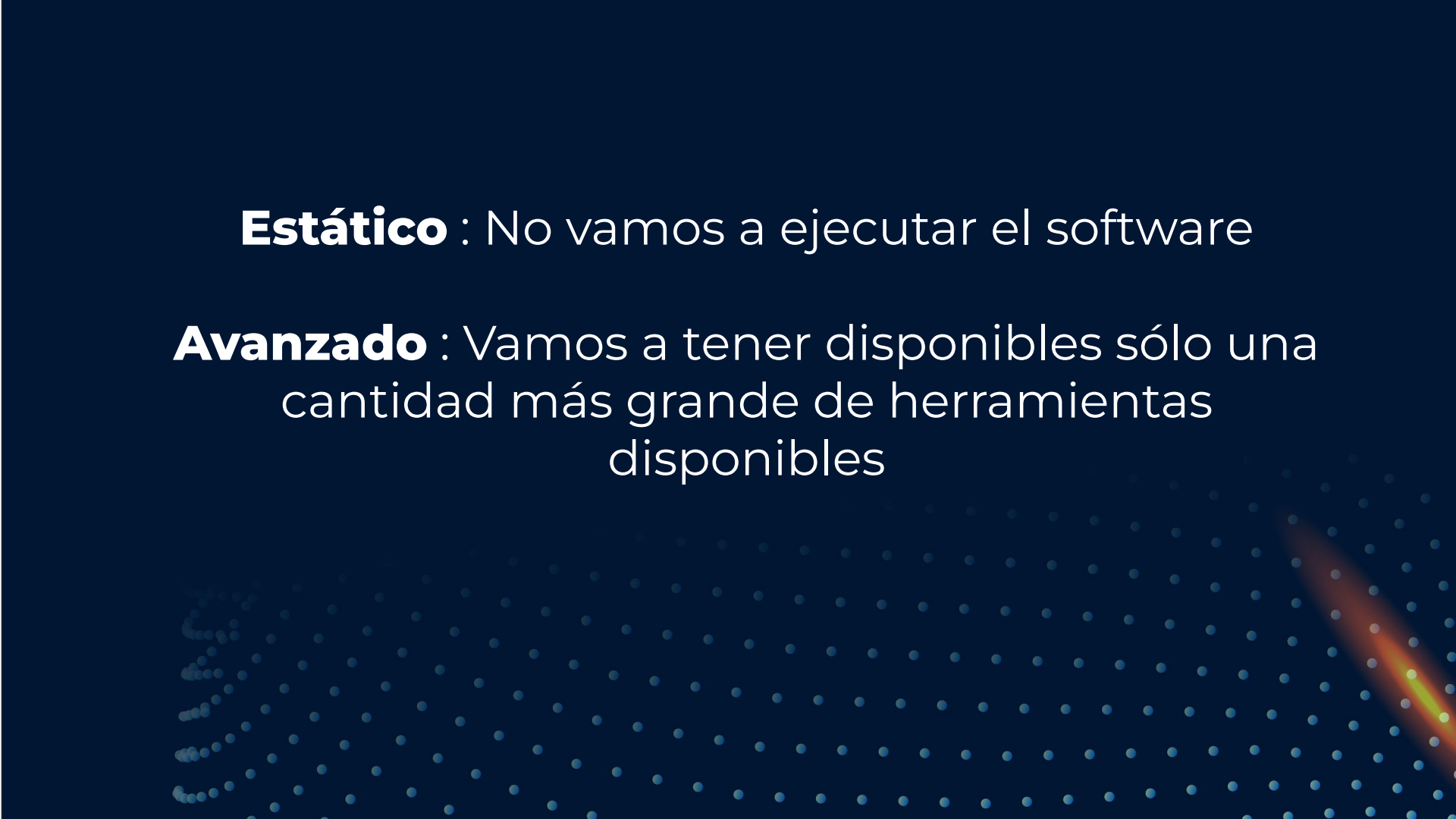


01 ANÁLISIS ESTÁTICO

Análisis Estático Avanzado

Estático : No vamos a ejecutar el software

Avanzado : Vamos a tener disponibles sólo una cantidad más grande de herramientas disponibles



El análisis dinámico es algo que se puede estar estudiando toda una vida y no aprenderse completamente.

El objetivo es obtener lo más cercano al código fuente del malware ya que los autores no son tan amables como para dejarlos disponible para los analistas

OBTENER EL CÓDIGO FUENTE



DESCOMPILAR

Código de alto nivel

DESENSAMBLAR

Código assembly



HISTORIA

- Hay cosas que los humanos entendemos y otras no.
- Se hizo una primera abstracción del código de máquina para que los humanos pudiéramos entender.
- Luego fueron siendo más entendibles los lenguajes.



EJECUCIÓN DE UN BINARIO x86

Instrucciones de CPU

Son el listado de instrucciones que le dicen al procesador que acción debe llevar adelante

Registers

Espacios de memoria de rápido acceso en donde se guarda información para realizar operaciones

Stack

El stack es una estructura de datos LIFO fundamental que se utiliza para gestionar llamadas a funciones, manejar interrupciones y evaluar expresiones.

INSTRUCCIONES DE CPU



Aritméticas

Realizan operaciones matemáticas como sub y add



Control

Llevan adelante el control del flujo como jmp, je y jnz



Movimiento

Mueven información de un lugar a otro como mov, push y pop



Otras

Otro tipo de instrucciones como call y ret. FLAGS.

REGISTERS

- **EAX**: Acumulador; usado para operaciones matemáticas y resultados.
- **EBX**: Base; puntero a datos en memoria.
- **ECX**: Contador; usado en bucles y repeticiones.
- **EDX**: Datos; usado para operaciones de entrada/salida.
- **ESI**: Índice fuente; usado para copiar datos.
- **EDI**: Índice destino; usado para copiar datos.
- **ESP**: Puntero de pila; apunta a la cima de la pila.
- **EBP**: Puntero base; apunta a la base de la pila.
- **EIP**: Puntero a la próxima instrucción.



STACK



Estructura

Estructura de datos LIFO que almacena información temporalmente.



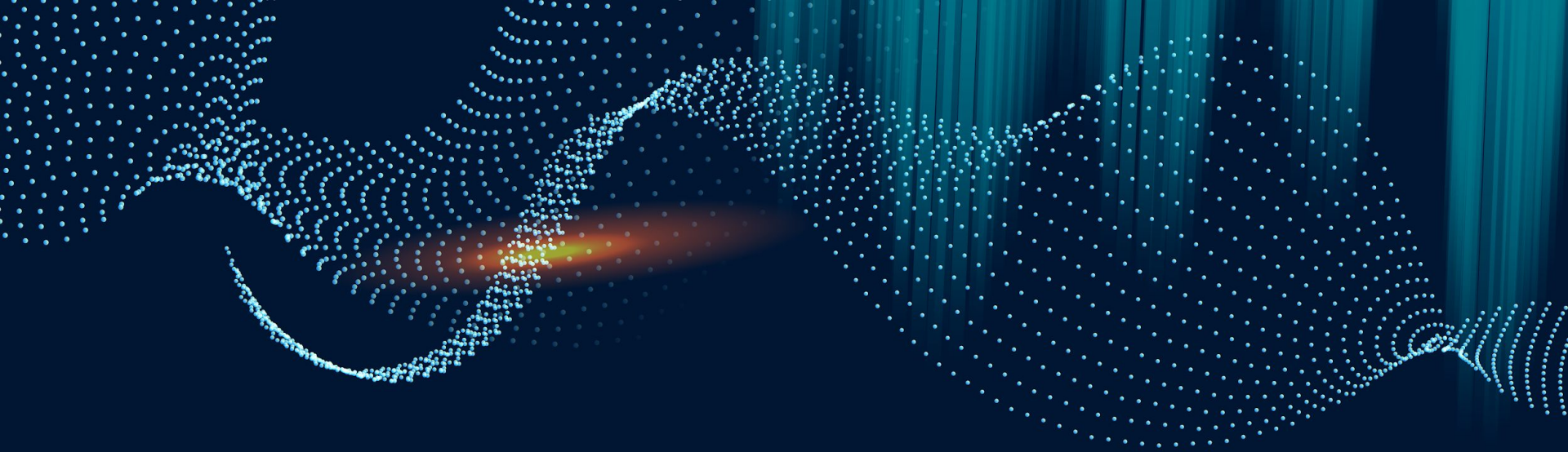
Tarea

Gestiona llamadas a funciones, interrupciones y evaluación de expresiones.



Maneja

Push agrega datos, **pop** los elimina en orden inverso al que fueron agregados.



02 ASSEMBLY

Introducción

GENERALIDADES

- Lenguaje ensamblador: lenguaje de programación de bajo nivel, representación legible por humanos del código de máquina
- Instrucciones: comandos fundamentales ejecutados por la CPU
- Instrucciones aritméticas: realizan operaciones matemáticas (p. ej., ADD, SUB, MUL, DIV)
- Instrucciones de flujo de control: administran la ejecución del programa (p. ej., JMP, JE, JNE, LOOP)
- Instrucciones de movimiento de datos: transfieren datos entre la memoria y los registros (p. ej., MOV, PUSH, POP)
- Otras instrucciones: CALL, RET, INT (interrupción)

MOVIMIENTO DE DATOS

	Descripción
MOV	Copia datos entre registros o memoria.
PUSH	Inserta un valor en la pila.
POP	Extrae un valor de la pila.

OPERACIONES ARITMÉTICAS

	Descripción
ADD	Suma dos valores.
SUB	Resta dos valores.
MUL	Multiplica dos valores.
DIV	Divide dos valores.

OPERACIONES LÓGICAS

	Descripción
AND	AND: Operación AND bit a bit.
OR	Operación OR bit a bit.
XOR	Operación XOR bit a bit.
NOT	Niega los bits de un valor.

SALTOS CONDICIONALES

	Descripción
JMP	Salto incondicional a una etiqueta.
JE	Salto si es igual.
JNE	Salto si no es igual.
JG	Salto si es mayor.
JL	Salto si es menor.

LLAMADAS A FUNCIONES

	Descripción
CALL	Llama a una función (subrutina).
RET	Regresa de una función.

¿QUÉ SON LOS REGISTROS?

- **Pequeñas unidades de memoria dentro del CPU.**
- **Almacenan datos y direcciones temporalmente.**
- **Acceso muy rápido.**

FLAGS

- Un registro especial de 32 bits dentro del CPU.
- Cada bit (bandera) representa un estado o condición particular del procesador.
- Se modifican automáticamente después de ejecutar instrucciones aritméticas o lógicas.

	Descripción
CF	Carry Flag: Indica acarreo o préstamo en operaciones aritméticas.
PF	Parity Flag: Indica si el número de bits en 1 en el resultado es par o impar.
AF	Auxiliary Carry Flag: Indica acarreo o préstamo entre los nibbles bajo y alto.
ZF	Zero Flag: Indica si el resultado de una operación es cero.
SF	Sign Flag: Indica si el resultado es negativo (bit de signo en 1).
OF	Overflow Flag: Indica desbordamiento en operaciones con signo.

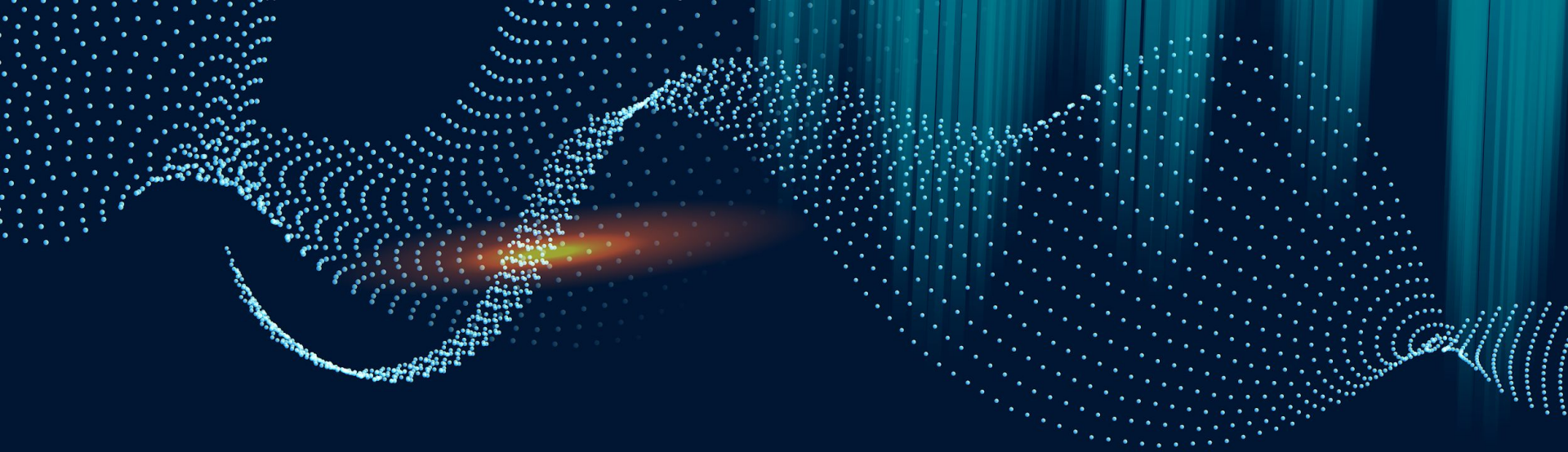
USOS

Son fundamentales para la ejecución de los programas

- **Toma de decisiones:** Las instrucciones de salto condicional (JE, JNE, JG, JL) se basan en el estado de las banderas para controlar el flujo del programa.
- **Ejemplo:** Después de ejecutar la instrucción ADD AX, BX, las banderas se actualizan según el resultado de la suma. Si el resultado es cero, la bandera ZF se establece en 1.

  Una instrucción JE (salto si es igual) saltaría a una etiqueta específica si ZF = 1.

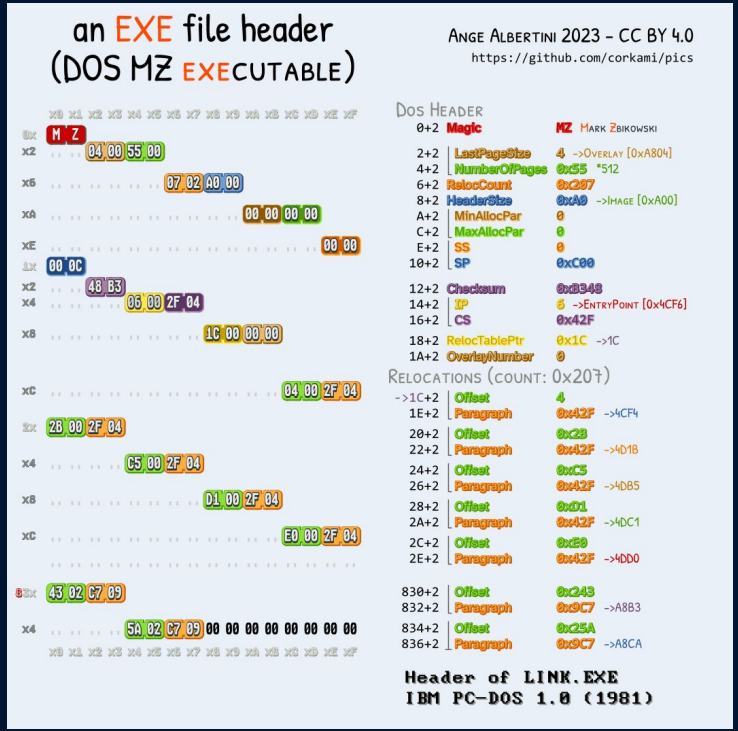
  Nota: Aunque el registro FLAGS tiene 32 bits, no todos se utilizan. Algunas banderas están reservadas para uso futuro o tienen funciones específicas del sistema operativo



03 ESTRUCTURA PE

En binario

ESTRUCTURA



https://github.com/corkami/pics/blob/master/binary/exe.png

ESTRUCTURA GENERAL



Encabezado DOS

Compatibilidad con sistemas DOS antiguos.



Encabezado PE

Indica el tipo de archivo (ejecutable, DLL, etc.) y la arquitectura del procesador.



Encabezado opcional

Información adicional, como la fecha de creación.

CASOS DE USO



Debugging

Entender cómo funciona un software



Seguridad

Identificar vulnerabilidad, fallas de sistemas, etc.



Interoperabilidad

Entender cómo se comunican dos productos



Compatibilidad

Mejorar la compatibilidad entre productos

Tabla de importación: Lista de funciones utilizadas desde DLLs externas.

¿Para qué sirve?

Permite que un ejecutable utilice funciones definidas en DLLs externas.

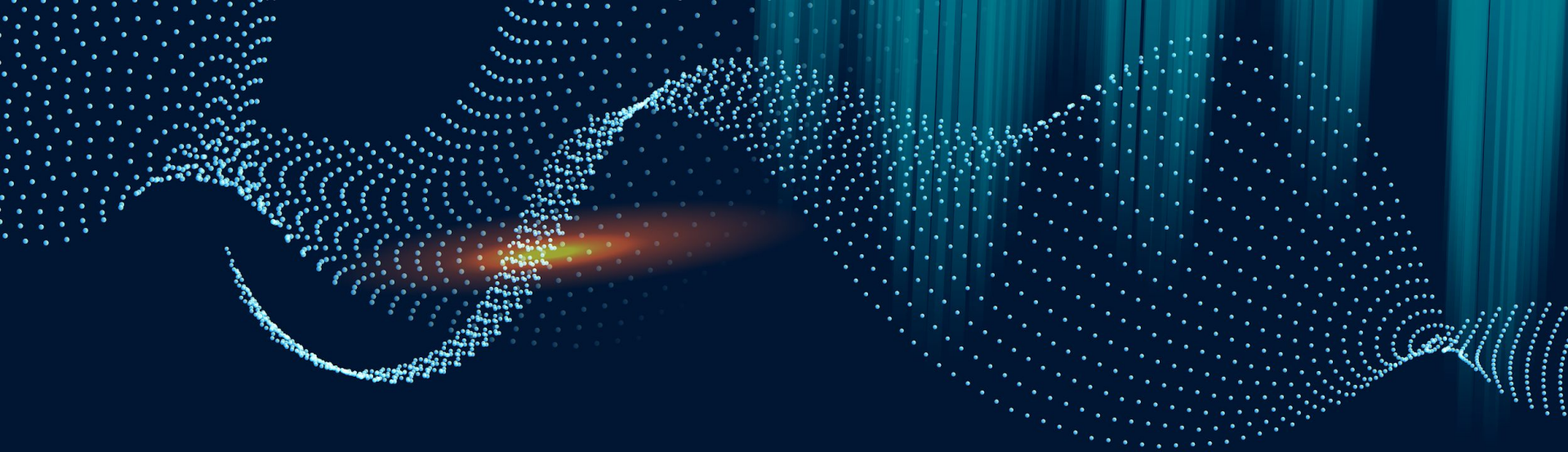
¿Cómo funciona?

Almacena una lista de DLLs y las funciones que se importan de cada una.

Durante la carga, el cargador dinámico resuelve las direcciones de las funciones importadas.

ENCABEZADO PE

- **ImageBase**: Dirección de memoria preferida para cargar el archivo.
- **EntryPoint**: Dirección de la primera instrucción a ejecutar.
- **SizeOfImage**: Tamaño total de la imagen en memoria.
- **NumberOfSections**: Número de secciones en el archivo.
- **Características**: Indica si el archivo es una DLL, una aplicación de consola, etc.
- **Machine**: Arquitectura del procesador (x86, x64, etc.).



Muchas gracias

Nos vemos en la próxima clase